

PlantasMon — Bitácora de Avance

Fecha: Mayo 2026 **Estudiante:** Brian Z. **Curso:** EIF209 Desarrollo y diseño de plataformas móviles **Profesor:** Daniel Granados Murillo **Universidad Nacional – Sede Regional Brunca** **Repositorio:** [GitHub — main branch](#)

Para exportar a PDF: Abrir este archivo en VS Code → Ctrl+Shift+P → "Markdown: Export to PDF". O usar línea de comandos: `npx md-to-pdf docs/Clase_Asincronica_Lab4_QA.md` Las capturas de pantalla deben estar en la carpeta `screenshots/` al mismo nivel que este archivo.

Tabla de Contenido

1. [Resumen Ejecutivo](#)
 2. [Bitácora por Módulo](#)
 - 2.1 [Autenticación \(Auth\)](#)
 - 2.2 [Home Dashboard](#)
 - 2.3 [API + Sincronización \(Lab3\)](#)
 - 2.4 [Explore](#)
 - 2.5 [Journal](#)
 - 2.6 [Profile](#)
 - 2.7 [Achievements y Rewards](#)
 - 2.8 [Missions y Rewards](#)
 - 2.9 [Popups Informativos](#)
 - 2.10 [Activity Feed](#)
 - 2.11 [Calendar](#)
 3. [Funcionalidad Propia — Achievements/Rewards](#)
 4. [Investigación de Testing/QA](#)
 5. [Pruebas Implementadas](#)
 6. [Repositorio](#)
 7. [Conclusiones](#)
-

1. Resumen Ejecutivo

PlantasMon es una aplicación móvil de cuidado de plantas desarrollada con **React Native / Expo SDK 54** que permite a los usuarios identificar plantas mediante la API de Pl@ntNet, llevar un registro de su colección, y ganar logros y recompensas a través de un sistema de gamificación. El backend utiliza **Firebase Firestore** para almacenamiento de datos y **Firebase Auth** para autenticación. La aplicación está desplegada localmente en Expo Go y el servidor API en **Render**.

La aplicación abarca desde la autenticación básica hasta un sistema completo de actividades, calendario, misiones, logros y popups informativos. Se implementaron **6 suites de pruebas** con **30 pruebas unitarias y de componente** que pasan al 100%, cubriendo validación de datos, servicios de actividad, detección de misiones y renderizado de pantallas críticas.

La aplicación está estructurada en capas: una capa de servicios que abstrae la lógica de negocio y acceso a Firestore; una capa de hooks que conecta servicios con componentes; y una capa de presentación con pantallas y componentes reutilizables.

La **funcionalidad propia** del proyecto es el sistema de **Achievements (Logros) y Rewards (Recompensas)**, un sistema de gamificación personalizado con 25 logros categorizados en 5 grupos, 30 objetos obtenibles con niveles de rareza (común, raro, épico, legendario), y un sistema de misiones diarias/semanales con recompensas en XP.

Stack Tecnológico

Capa	Tecnología	Propósito
Framework	Expo SDK 54 + React Native 0.81	Desarrollo multiplataforma iOS/Android
Autenticación	Firebase Auth	Email/password + Google OAuth
Base de datos	Firebase Firestore	Usuarios, plantas, actividades, logros
API Externa	Pl@ntNet API	Identificación de plantas por imagen
Servidor API	Node.js / Express (Render)	Proxy para Pl@ntNet API
Offline	AsyncStorage	Caché local + cola de sincronización
Testing	Jest + React Native Testing Library	Pruebas unitarias y de componente
Navegación	expo-router (file-based)	Enrutamiento declarativo
Calendario	react-native-calendars	Marcadores de eventos por día

2. Bitácora por Módulo

2.1 Autenticación (Auth)

% Avance	Decisiones Clave	Problemas Encontrados y Soluciones
100%	Login email/password con Firebase Auth	Google OAuth requería fix de credential exchange — se corrigió con <code>signInWithCredential</code>
	Google OAuth con expo-auth-session	Register screen no existía inicialmente — se creó formulario de registro
	Auth persistence con <code>onAuthStateChanged</code>	IDs hardcoded (<code>u_001</code>) en Home y Profile — se migró a dynamic user ID
	Dynamic user ID migration completa	

Captura de pantalla: `screenshots/auth-login.png` Captura de pantalla: `screenshots/auth-register.png`

2.2 Home Dashboard

% Avance	Decisiones Clave	Problemas Encontrados y Soluciones
100%	Dashboard personalizado por usuario autenticado	Datos hardcoded reemplazados con Firebase Auth UID real
	StatsBar: account age, fotos hoy, racha de riego, última identificación	PlantOfTheDay diferido (sin endpoint listo)

	Misiones diarias/semanales visibles en Home	
	Timeline de actividad real desde Firestore	

Captura de pantalla: screenshots/home-dashboard.png **Captura de pantalla:** screenshots/home-missions.png

2.3 API + Sincronización (Lab3)

% Avance	Decisiones Clave	Problemas Encontrados y Soluciones
100%	Pl@ntNet API para identificación de plantas	API desplegada en Render (cloud hosting requerido)
	Offline support con local storage + sync queue	Offline fallback en identify screen
	Module analysis para capacidad offline	

Captura de pantalla: screenshots/identify-plant.png

2.4 Explore

% Avance	Decisiones Clave	Problemas Encontrados y Soluciones
100%	Catálogo de plantas desde Firestore con cuadrícula de 2 columnas	Búsqueda client-side case-insensitive implementada
	Navegación a detalle de planta vía plant/[id]	

Captura de pantalla: screenshots/explore-grid.png

2.5 Journal

% Avance	Decisiones Clave	Problemas Encontrados y Soluciones
100%	Dashboard basado en cards con plantas analizadas	Se reutilizaron componentes DailyMissions + UserProgress
	Secciones de misiones, progreso y logros	Pull-to-refresh en Explore y Journal

Captura de pantalla: screenshots/journal-dashboard.png

2.6 Profile

% Avance	Decisiones Clave	Problemas Encontrados y Soluciones
----------	------------------	------------------------------------

100%	Edición de perfil (nombre, bio, ubicación)	Datos reales desde Firestore reemplazaron mock ACTIVITIES
	Achievements display con locked/unlocked visual	Achievements seeded en Firestore
	Activity feed con datos reales desde Firestore (useActivityFeed)	

Captura de pantalla: screenshots/profile-vitrina.png **Captura de pantalla:** screenshots/profile-activity.png

2.7 Achievements y Rewards (Funcionalidad Propia)

% Avance	Decisiones Clave	Problemas Encontrados y Soluciones
100%	25 achievements definidos y seedeados en Firestore	Client-side unlock logic implementada
	XP rewards al desbloquear logros	rewardItemId para futuros item grants
	Estados visuales locked/unlocked en Profile	

Nota: Este es el módulo de **funcionalidad propia** — ver sección dedicada más abajo.

2.8 Missions y Rewards

% Avance	Decisiones Clave	Problemas Encontrados y Soluciones
100%	Rotación diaria (5) + semanal (2) de misiones	Rotación basada en seed determinística
	25 misiones diarias + 10 semanales definidas en Firestore	Progress tracking multi-etapa
	Flujo tap-to-claim con recompensa XP	Grace period para misiones no reclamadas
	Detección de completitud basada en eventos (identify, water, scan, share)	Objetos obtenibles (30 items, 4 rarezas)
	Profile vitrina para items obtenidos	

Captura de pantalla: screenshots/missions-daily.png

2.9 Popups Informativos

% Avance	Decisiones Clave	Problemas Encontrados y Soluciones
100%	Popups tipo bottom-sheet (InfoBottomSheet)	Persistencia de dismiss por usuario (usePopupDismissal)

	Popups de celebración en unlocks (CelebrationSheet)	Detección de primer uso por popupKey
	Iconos de información on demand en varias pantallas	

Captura de pantalla: `screenshots/popup-celebration.png`

2.10 Activity Feed

% Avance	Decisiones Clave	Problemas Encontrados y Soluciones
100%	Subcolección Firestore <code>users/{uid}/activities</code>	Dependencia circular resuelta con <code>require()</code> inline
	4 tipos de evento: water, identify, achievement, mission	Firestore Timestamp a Date parsing
	Formato de tiempo relativo en español	

2.11 Calendar

% Avance	Decisiones Clave	Problemas Encontrados y Soluciones
100%	react-native-calendars con multi-dot markers	Colores de dot por tipo de evento
	4 colores: water (azul), identify (verde), achievement (oro), mission (púrpura)	Locale español para meses y días
	Day tap muestra lista de eventos via ActivityFeed	

Captura de pantalla: `screenshots/calendar-view.png`

3. Funcionalidad Propia — Achievements/Rewards

El sistema de **Achievements (Logros) y Rewards (Recompensas)** constituye la funcionalidad propia del proyecto — un sistema de gamificación personalizado que va más allá de los requisitos básicos del laboratorio.

Valor para el Usuario

Este sistema transforma una aplicación utilitaria de cuidado de plantas en una experiencia gamificada que motiva al usuario a:

- **Regresar diariamente** para completar misiones y mantener su racha
- **Identificar más plantas** para desbloquear logros de milestone
- **Cuidar consistentemente** sus plantas para obtener recompensas
- **Coleccionar objetos** con rarezas para decorar su vitrina de perfil

Arquitectura

Modelo de datos en Firestore:

```
achievements/                                ← Colección de definiciones de logros
{ achievementId }:
  id, title, description, icon, category
  requirement: { type, count }
  xpReward, stages?, rewardItemId?

users/{uid}/achievements/                    ← Subcolección de progreso por usuario
{ achievementId }:
  current, unlockedAt?, claimed
```

Flujo de detección de desbloqueo:

- 1. Usuario realiza acción (identificar, regar, completar misión)
- 2. `checkAndUnlockAchievements()` se ejecuta
- 3. Obtiene definiciones de logros + progreso actual del usuario
- 4. Para cada logro matching: incrementa contador, verifica si alcanzó la meta
- 5. Si se desbloqueó: registra timestamp, otorga XP, registra evento en Activity Feed, muestra CelebrationSheet

Categorías de Logros (25 total)

Categoría	Cantidad	Descripción	Ejemplos
Milestone	8	Hitos por plantas identificadas	1, 5, 10, 25, 50, 100 plantas
Streak	5	Racha de días consecutivos	3, 7, 14, 30, 100 días
Account Age	4	Antigüedad de la cuenta	7d, 30d, 100d, 1 año
Weekly Active	5	Semanas completas	1, 4, 8, 12, 24 semanas
Social	3	Interacciones (futuro)	Compartir, referir amigos

Sistema de Recompensas

Los logros otorgan XP que se acumulan en el perfil (50 XP simples, hasta 500 XP para logros de larga duración). El campo `rewardItemId` está definido para futura concesión automática de objetos al completar logros (actualmente los 30 objetos obtenibles se obtienen mediante misiones).

Integración Transversal

Módulo	Integración
Profile	Lista de logros con filtro locked/unlocked
Home Dashboard	Resumen de progreso en StatsBar
Missions	Logro semanal "Completa todas las misiones"
Activity Feed	Evento "achievement" registrado al desbloquear
Popups	CelebrationSheet en desbloques
Calendar	Marcador dorado en días con logro desbloqueado

4. Investigación de Testing/QA

4.1 Herramientas Comparadas

Herramienta	Tipo	Setup en Expo	Velocidad	Comunidad	Cobertura
Jest + React Native Testing Library	Unit + Component	✅ Fácil (jest-expo preset)	⚡ Rápida	Grande, madura	Lógica + UI con mocks
Detox	E2E	❌ Complejo (requiere build iOS/Android)	🐢 Lenta	Mediana	Flujo completo real
Maestro	E2E	⚠️ Medio (requiere app build)	⚡ Rápida	Creciente	Flujo completo real

4.2 Evaluación

- **Setup:** Jest+RTL gana con preset `jest-expo` de una línea
- **Velocidad:** Jest+RTL corre en Node.js sin dispositivo
- **Tipo necesario:** Unit + Component es suficiente para el alcance actual
- **Recursos:** Comunidad más grande, estándar de la industria
- **Mocks:** Jest permite mockear Firebase, react-native-calendars, expo-router fácilmente

4.3 Conclusión

Recomendación: Jest + React Native Testing Library. Setup fácil en Expo, ejecución rápida, cobertura suficiente. E2E investigado pero no implementado — recomendado para cuando la app sea más estable.

4.4 Configuración de Jest

```
module.exports = {
  preset: "jest-expo",
  setupFilesAfterEnv: ["<rootDir>/jest.setup.ts"],
  moduleNameMapper: { "^.*/(.*)$": "<rootDir>/$1" },
  testMatch: ["**/__tests__/**/*.test.(ts|tsx)"],
  transformIgnorePatterns: [
    "node_modules/(?!(jest-)?react-native|@react-native|expo(nent)?|expo-router|@expo(nent)?/)"
  ]
}
```

4.5 Patrones de Mock

Tres patrones usados para aislar tests de dependencias nativas:

Firestore Mock:

```
jest.mock("@/src/config/firebase", () => ({ db: {} }));
jest.mock("firebase/firestore", () => ({
```

```
doc: jest.fn(), getDoc: jest.fn(), setDoc: jest.fn(),
collection: jest.fn(), Timestamp: { now: () => ({ toDate: () => new Date() }) },
serverTimestamp: jest.fn(),
}));
```

Módulos Nativos:

```
jest.mock("react-native-safe-area-context", () => ({
  useSafeAreaInsets: () => ({ top: 0, bottom: 0, left: 0, right: 0 }),
  SafeAreaView: ({ children }: any) => children,
}));
```

Servicios Mockeables:

```
const mockGetUserActivities = jest.fn();
jest.mock("@/src/services/activityService", () => ({
  getUserActivities: (...args: any[]) => mockGetUserActivities(...args),
}));
```

5. Pruebas Implementadas

5.1 Test Unitario — Validación de Planta

- **Archivo:** `src/lib/__tests__/plantValidation.test.ts`
- **Propósito:** Validar que `validatePlantData()` acepta datos correctos y rechaza inválidos con mensajes en español

```
describe("validatePlantData", () => {
  it("acepta datos válidos", () => {
    const result = validatePlantData({
      commonName: "Monstera",
      scientificName: "Monstera deliciosa",
      wateringDays: 7, sunlight: "partial",
    })
    expect(result.valid).toBe(true)
    expect(result.errors).toEqual([])
  })

  it("reporta errores cuando hay datos inválidos", () => {
    const result = validatePlantData({
      commonName: "", scientificName: "A",
      wateringDays: 0, sunlight: "shade" as any,
    })
    expect(result.valid).toBe(false)
    expect(result.errors).toEqual(expect.arrayContaining([
      "El nombre común debe tener al menos 2 caracteres",
      "El nombre científico debe tener al menos 3 caracteres",
      "El riego debe estar entre 1 y 30 días",
      "La luz debe ser low, partial o full",
    ]))
  })
})
```



```
    ]))
  })
})
```

- **Resultado:** ✅ PASA — 2 tests, 0 failures
- **Captura de terminal:** `screenshots/test-plant-validation.png`

5.2 Test Unitario — Activity Service Helpers

- **Archivo:** `src/services/__tests__/activityService.test.ts`
- **Propósito:** Probar funciones puras: `formatRelativeTime` (formato español), `safeParseDate` (null/undefined/Timestamp/Date), `toActivityData` (mapeo de tipos)

```
// Casos clave:
it("devuelve 'Ahora' cuando la fecha es el mismo instante", () => { ... })
it("devuelve 'Hace 5 min' cuando pasaron 5 minutos", () => { ... })
it("devuelve 'Hace 1 h' cuando pasó 1 hora", () => { ... })
it("devuelve fecha formateada (dd/mm/yyyy) cuando pasaron 7+ días", () => { ... })
it("parsea un string ISO a Date", () => { ... })
it("mapea un evento 'identify' correctamente", () => { ... })
it("mapea un evento 'mission' como 'achievement'", () => { ... })
```

- **Resultado:** ✅ PASA — 17 tests, 0 failures
- **Captura de terminal:** `screenshots/test-activity-service.png`

5.3 Test de Componente — LoginScreen

- **Archivo:** `src/screens/auth/__tests__/loginScreen.test.tsx`
- **Propósito:** Verificar que presionar "Entrar" con formulario vacío muestra error

```
jest.mock("expo-router", () => ({
  useRouter: () => ({ push: jest.fn() }),
}))
jest.mock("react-native-safe-area-context", () => ({
  useSafeAreaInsets: () => ({ top: 0, bottom: 0, left: 0, right: 0 }),
  SafeAreaView: ({ children }: any) => children,
}))

describe("LoginScreen", () => {
  it("muestra error cuando el formulario está vacío", () => {
    const { getByText, queryByText } = render(<LoginScreen />)
    expect(queryByText("Por favor completa todos los campos.")).toBeNull()
    fireEvent.press(getByText("Entrar"))
    expect(getByText("Por favor completa todos los campos.")).toBeTruthy()
  })
})
```

- **Resultado:** ✅ PASA — 1 test, 0 failures
 - **Captura de terminal:** `screenshots/test-login-screen.png`
-

5.4 Test de Componente — CalendarScreen

- **Archivo:** src/screens/calendar/__tests__/CalendarScreen.test.tsx
- **Propósito:** Verificar renderizado del calendario: título español, leyenda colores, estados vacío/carga

```
it("muestra indicador de carga al iniciar", () => { ... })
it("muestra calendario y mensaje vacío cuando no hay actividades", async () => { ... })
it("muestra leyenda de colores en español", async () => { ... })
it("muestra el título en español", () => { ... })
```

- **Resultado:** PASA — 4 tests, 0 failures
- **Captura de terminal:** screenshots/test-calendar-screen.png

5.5 Tests Adicionales

missionService.test.ts — Grace period y asignación de misiones (2 tests):

- Verifica que assignDailyMissions registra assignedDate en nuevas entradas
- Verifica que getExpiredMissions solo retorna misiones completadas+no reclamadas del día anterior
- Resultado: PASA

useMissionProgress.test.ts — Hook de progreso de misiones (4 tests):

- Exporta reportMissionProgress como función async standalone
- Obtiene misiones del usuario y actualiza progreso
- Maneja errores gracefully sin lanzar excepciones
- Hook useMissionProgress retorna reportProgress que comparte lógica
- Resultado: PASA

5.6 Resultado General de Tests

Ejecutado el 24 de Mayo 2026:

```
PASS src/lib/__tests__/plantValidation.test.ts
PASS src/services/__tests__/activityService.test.ts
PASS src/services/__tests__/missionService.test.ts
PASS src/hooks/__tests__/useMissionProgress.test.ts
PASS src/screens/calendar/__tests__/CalendarScreen.test.tsx
PASS src/screens/auth/__tests__/loginScreen.test.tsx

Test Suites: 6 passed, 6 total
Tests:      30 passed, 30 total
Snapshots:  0 total
Time:       9.011 s
Ran all test suites.
```

Test Suite	Tipo	Tests	Estado
plantValidation.test.ts	Unit	2	PASA
activityService.test.ts	Unit	17	PASA

loginScreen.test.tsx	Component	1	✓ PASA
CalendarScreen.test.tsx	Component	4	✓ PASA
missionService.test.ts	Unit	2	✓ PASA
useMissionProgress.test.ts	Hook	4	✓ PASA
Total		30	✓ TODOS PASAN

Captura de terminal general: screenshots/test-all-passing.png

5.7 Reflexión sobre la Experiencia de Testing

Integrar pruebas en un proyecto ya existente resultó ser un desafío significativo, principalmente por la dependencia de módulos nativos de React Native y Firebase que requirieron un sistema de mocks extenso para ejecutar los tests en Node.js sin un dispositivo físico. Lo más complejo fue configurar correctamente los mocks para `expo-router`, `react-native-safe-area-context`, y los servicios de Firebase, ya que cualquier mock faltante rompía toda la suite. Si hubiera pensado en pruebas desde el inicio, habría estructurado mejor la separación entre lógica de negocio y componentes de UI — extrayendo funciones puras (como `formatRelativeTime` o `safeParseDate`) a módulos independientes desde el principio. También habría establecido un patrón de inyección de dependencias para Firebase, permitiendo reemplazar la base de datos real con mocks en tests de componentes. Una vez que la infraestructura de mocks estuvo lista, agregar nuevos tests fue directo — los 17 tests del `activityService` se escribieron en minutos porque las funciones puras no requieren mock. La lección principal: la testabilidad debe ser un requisito arquitectónico desde el inicio, no algo que se agrega al final.

6. Repositorio

- **URL:** <https://github.com/Z3E-Brian/Plantasmon>
- **Branch:** main

Para generar el PDF final:

```
# Opción 1: VS Code → Ctrl+Shift+P → "Markdown: Export to PDF"
# Opción 2: Línea de comandos
npx md-to-pdf docs/Clase_Asincronica_Lab4_QA.md
```

Nota: Las capturas de pantalla referenciadas deben estar en la carpeta `screenshots/` al mismo nivel que `BITACORA.md`. Se generan desde la app en Expo Go y desde la terminal de pruebas.

7. Conclusiones

El desarrollo de PlantasMon ha demostrado la viabilidad de construir una aplicación completa de cuidado de plantas con React Native/Expo, integrando Firebase Auth, Firestore y la API de Pl@ntNet.

Logros principales:

- Autenticación completa (email/password + Google OAuth + registro)
- Identificación de plantas mediante API con soporte offline
- Catálogo explorable con búsqueda y journal dashboard
- Dashboard personalizado con estadísticas, misiones y timeline

- **Funcionalidad propia:** Sistema de gamificación con 25 logros, misiones diarias/semanales, 30 objetos obtenibles con rarezas, y popups de celebración
- Calendario de actividades con marcadores por color
- **Infraestructura de testing:** 6 suites, 30 tests, 100% pasando

Desafíos encontrados:

- Google OAuth requirió fix de credential exchange
- Migración de IDs hardcoded a IDs dinámicos de Firebase Auth
- Sincronización offline con cola de operaciones
- Dependencia circular resuelta con patrón require()
- Parseo de Firestore Timestamp a Date para activity feed

Próximos pasos:

- Push notifications para recordatorios de riego
- E2E testing con Detox o Maestro
- Diagnóstico de enfermedades de plantas
- Widgets para home screen del dispositivo
- Modo offline completo con Firestore persistence

Documento generado el 24 de Mayo de 2026 para el curso EIF209.